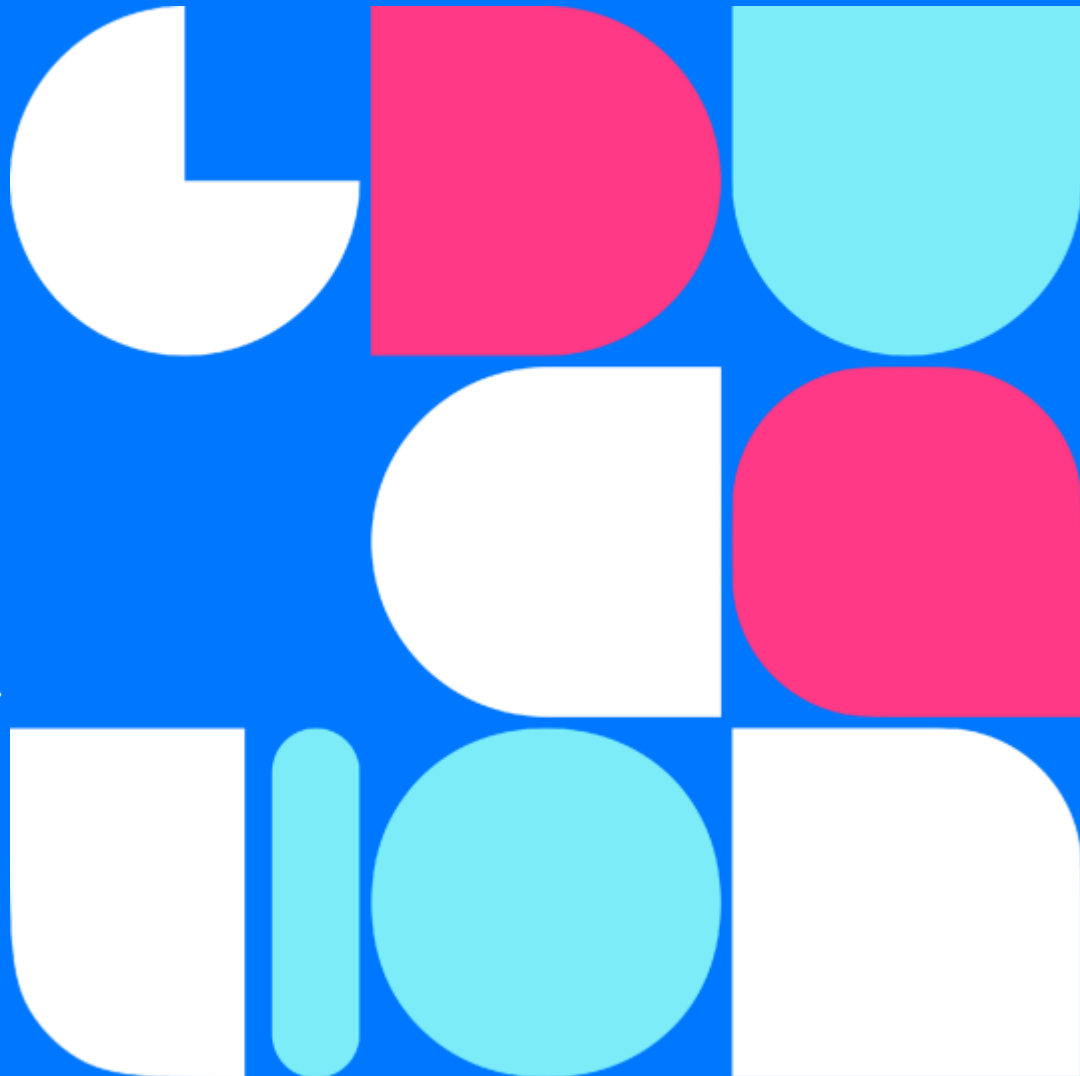
education

Текстовое ранжирование. Часть I

«Информационный поиск и ранжирование»

Преподаватель Руслан Аллаяров



Ведущий лекции



- Более 7 лет в ML, 6 лет из них в поиске
- Lead developer (реком товаров Ozon), Ex VK (поиск), Ex ОККО (реком главной витрины)
- Отвечал за блендинг поиска по видео, за базовое качество поиска по клипам и плейлистам
- Закончил мехмат МГУ и Техносферу (2021)

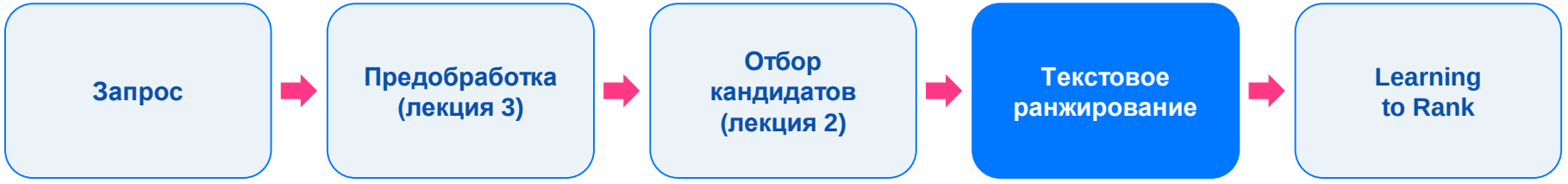
education

План лекции

1. Задача ранжирования
2. Как сравнивать ранжирования
3. TF-IDF
4. Модель векторного пространства
5. Ранжирование на обратном индексе
6. Лексический разрыв
7. Итоги и материалы
8. Семинар

Что мы уже знаем

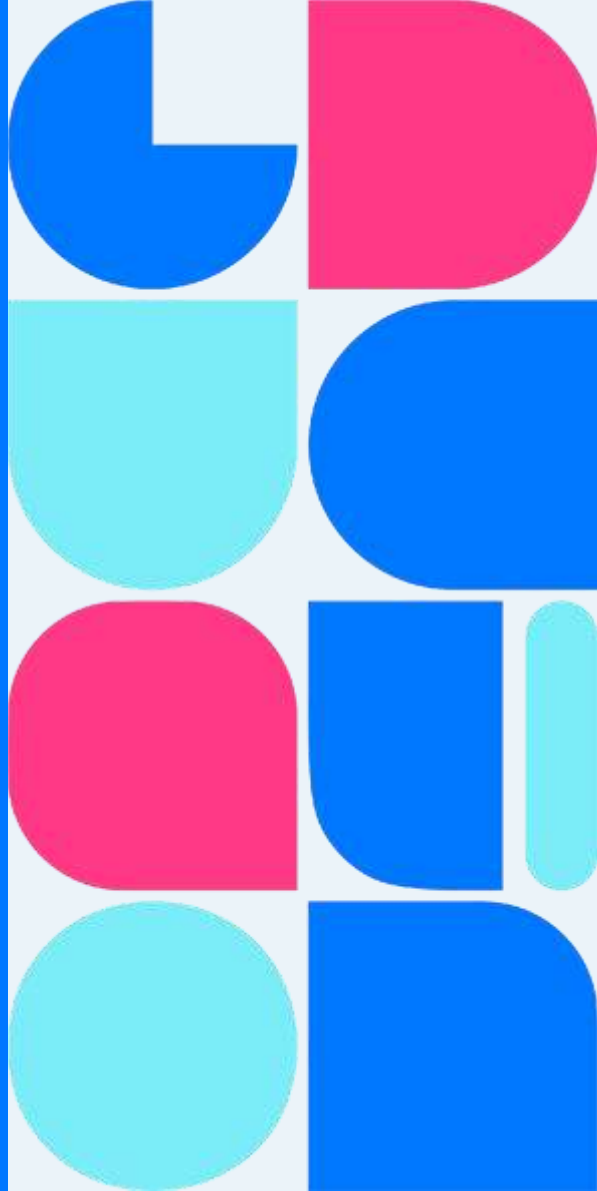
- **Лекция 1:** пирамида ранжирования, мягкое И, TF-IDF и BM25 — на уровне идеи
- **Лекция 2:** обратный индекс, постинг-листы, координатный индекс, лемматизация, закон Ципфа
- **Лекция 3:** предобработка запроса, n-граммные языковые модели и сглаживание
- Сегодня собираем из этого первую настоящую формулу ранжирования



сегодняшняя лекция — про четвёртый блок: как отсортировать кандидатов, имея только тексты

Задача ранжирования

Вспомнить, зачем нужен скор Score(q, d) и где в поиске живёт текстовое ранжирование



Ad hoc поиск

- Булев поиск хорош для экспертов, но не для обычных пользователей
  - Никто не пишет логические выражения
  - Строгое И слишком жёсткое, ИЛИ — слишком мягкое
- Ad hoc поиск: запрос в свободной форме + ранжирование результатов
- Пользователь смотрит только верх выдачи, поэтому главное — порядок

Запрос из 12 слов

«для решения каких задач машинного обучения лучше всего подходит метод опорных векторов»

Как строгий И-запрос почти ничего не найдёт. Нужно ранжировать по степени совпадения.

Повторение из лекции 1

Ранжирование = сортировка по скору

Каждой паре (запрос, документ) назначаем число Score(q, d) и сортируем документы по убыванию

$$\text{выдача}(q) = \text{sort_desc}\{\, d \in \text{кандидаты}(q) : \text{Score}(q, d) \,\}$$

Скор — функция от пары, а не от документа: по разным запросам один и тот же документ стоит на разных местах

Где используется текстовый скор

1. Самостоятельно: поиск без ML (whoosh, Lucene «из коробки»)
2. Черновое ранжирование: отобрать топ-K кандидатов
3. Признак для Learning to Rank

Почему это важно в 2026

1. Lucene/Elasticsearch с 2016 года по умолчанию ранжируют BM25
2. RAG и агенты вызывают обычный поиск
3. Лексический скор — сильный бейзлайн для нейросетей

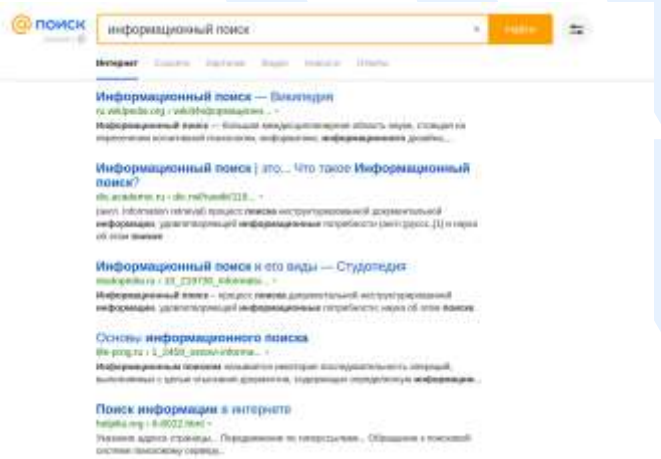
Пирамида ранжирования: мы тут

Learning to Rank
~10^2 → топ-10

Черновое ранжирование: TF-IDF, BM25
~10^4 → ~10^2

Отбор кандидатов: мягкое И / кворум
~10^6 → ~10^4

Обратный + векторный индекс
~10^6 документов



Сегодня: второй и третий снизу уровни.

Скор считаем только для кандидатов, поднятых из обратного индекса.

Повторение из лекции 1

Постановка задачи

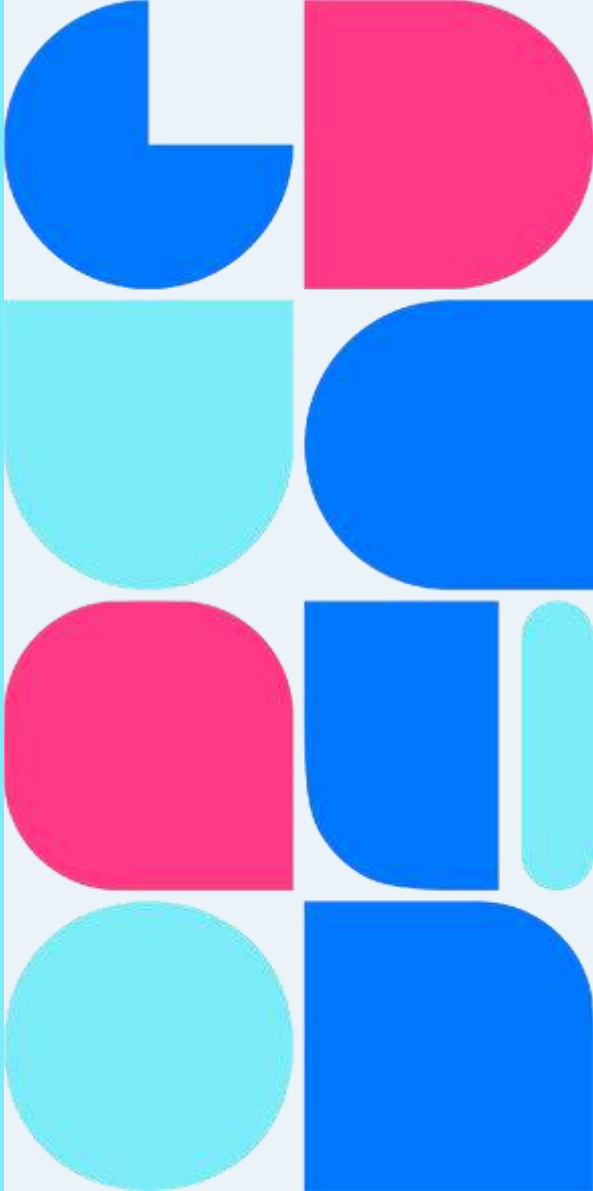
- Дано: корпус документов D, поток запросов q
- Нужно: функция Score(q, d), которая ставит релевантные документы выше нерелевантных
- Есть только тексты запроса и документа — ни кликов, ни ссылок, ни ML
- Такое ранжирование называют **текстовым** (lexical matching)
- Сегодня: эвристические и векторные модели. Через неделю: вероятностные (BM25)

Главный вопрос лекции

Как из «слова запроса нашлись в документе» сделать число, по которому сортировка будет правильной — и как это посчитать за миллисекунды?

Как сравнивать ранжирования

Прежде чем улучшать формулу, научиться измерять её качество



Тестовая коллекция

Парадигма Cranfield (1960-е) и TREC (с 1992): качество измеряют на фиксированном наборе данных

Документы	Запросы	Оценки (qrels)
корпус, по которому ищем	выборка реальных или типовых запросов	для пар (q, d): релевантен или нет

- Оценки ставят асессоры, всё чаще — LLM (подробнее в части II)
- Разметить все пары невозможно: оцениваем только верх выдач разных систем (pooling)
- Разметили один раз → любой вариант формулы проверяем офлайн за минуты, а не A/B-тестом на пользователях

На семинаре: RuBQ — вопросы к русской Википедии, 1692 запроса с разметкой

Метрики на топе выдачи

Пользователь видит первые K документов, поэтому метрики считаем «@K»

Метрика	Формула	Что показывает
Precision@K	|рел. в топ-K| / K	насколько чистый топ
Recall@K	|рел. в топ-K| / |все рел.|	сколько нужного нашли
RR (reciprocal rank)	1 / позиция первого релевантного	как быстро найдётся ответ
MRR@K	среднее RR по запросам (0, если нет в топ-K)	для «одного правильного ответа»

Метрики всегда усредняют по запросам. MAP и nDCG с градуированной релевантностью — в части II

Считаем на примере

Выдача по запросу, в коллекции всего 3 релевантных документа

#1	#2	#3	#4	#5	#6	#7	#8	#9	#10
—	рел.	—	рел.	—	—	—	рел.	—	—

P Precision@5 = 2 / 5 = 0.4

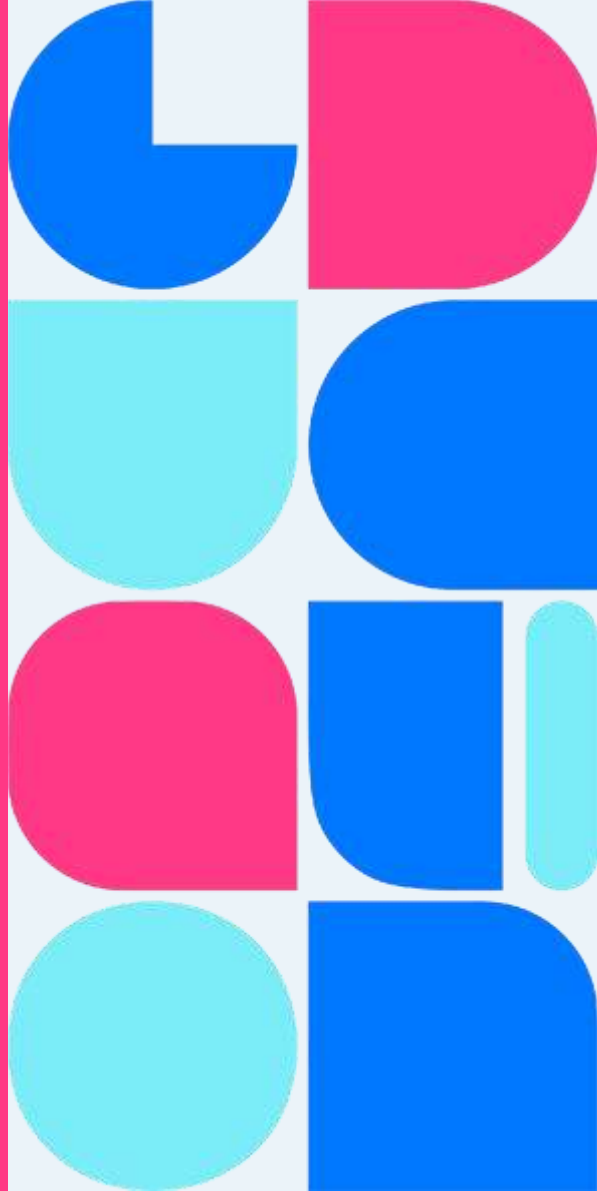
R Recall@5 = 2 / 3 ≈ 0.67, Recall@10 = 3 / 3 = 1.0

RR RR = 1 / 2 = 0.5 (первый релевантный на позиции 2)

! Если бы первый релевантный был на 12-й позиции: RR@10 = 0

TF-IDF

Собрать первую формулу ранжирования из двух наблюдений о словах



Термины

- Термин — единица индексирования: нормализованное слово
- Нормализация: регистр + лемматизация (или стемминг) + иногда стоп-слова
- Запрос и документ обрабатываем **одинаково**
- Дальше «слово» и «термин» — синонимы, термины пишем КАПСОМ

Пример

«участники международного конкурса юных музыкантов»



[УЧАСТНИК, МЕЖДУНАРОДНЫЙ, КОНКУРС, ЮНЫЙ, МУЗЫКАНТ]

Повторение из лекции 2

TF: частота термина

- Начнём с однословных запросов, например «физтех»
- TF(t, d) — сколько раз термин t встречается в документе d
- Гипотеза: чем чаще слово запроса в документе, тем документ релевантнее
- Для многословного запроса просто сложим:

$$\mathrm{TfScore}(q, d) = \sum \mathrm{TF}(t, d), \quad t \in q$$

Запрос «физтех»

Википедия про МФТИ: ФИЗТЕХ встречается 107 раз
Википедия про город Долгопрудный: 3 раза

Проблема 1: keyword stuffing

Welcome to Red Apples Online!

If red apples are what you're looking for, then you are definitely in the right place to buy red apples. When it comes to red apples, you won't find a higher quality selection of red apples anywhere! Our red apples experts know how to pick only the best, most savory red apples from the bunch, and we sell these premium red apples right here for you to enjoy (red apples). Seriously, go to another red apples website and try their red apples. We guarantee you'll come crawling back to buy our red apples, buddy.

Так выглядит keyword stuffing: TfScore растёт линейно, спамеру это выгодно

- TF растёт линейно — 50 повторов «весят» в 50 раз больше одного
- Но первое вхождение говорит о теме гораздо больше, чем 50-е
- Нужно насыщение: каждое следующее вхождение даёт меньше

Сублинейный TF

$$wf = 1 + \log TF, \quad \text{если } TF > 0$$

Проблема 2: слова неравноценны

Запрос «институт итмо». У двух документов одинаковый TfScore = 1

Московский физико-технический институт

совпало слово ИНСТИТУТ — есть в тысячах документов

Университет ИТМО

совпало слово ИТМО — редкое, почти однозначно указывает на тему

- Чем реже слово в коллекции, тем оно информативнее
- Крайний случай — стоп-слова (И, В, НА): встречаются везде и почти ничего не значат
- «административная ответственность за собакокрадство»: вся информация в последнем слове

DF и IDF

- DF(t) — в скольких документах встречается термин = длина его постинг-листа
- IDF — обратная документная частота, N — число документов:

Классика

$$\mathrm{IDF}(t) = \log(N / \mathrm{DF}(t))$$

Сглаженная (scikit-learn)

$$\mathrm{IDF}(t) = 1 + \ln((N+1) / (\mathrm{DF}(t)+1))$$

Термин	DF	IDF = ln(3 / DF)
ИТМО	1	1.10
МОСКОВСКИЙ	2	0.41
УНИВЕРСИТЕТ	2	0.41

Корпус: МФТИ, МГУ, ИТМО

IDF — свойство всей коллекции, а не конкретного документа

Почему логарифм — выведем в части II

TF-IDF

$$\mathrm{TfIdf}(q, d) = \Sigma\, \mathrm{TF}(t, d) \cdot \mathrm{IDF}(t), \quad t \in q \cap d$$

- TF — локальная статистика: насколько слово важно в этом документе
- IDF — глобальная: насколько слово информативно вообще
- Суммируем только по совпавшим терминам: остальные дают 0
- Работает и для однословных, и для многословных запросов



Карен Спарк Джонс (1935–2007) предложила IDF в 1972 году: «A statistical interpretation of term specificity...»

Считаем на примере

Запрос «университет итмо», корпус из трёх документов, TF = 1 везде

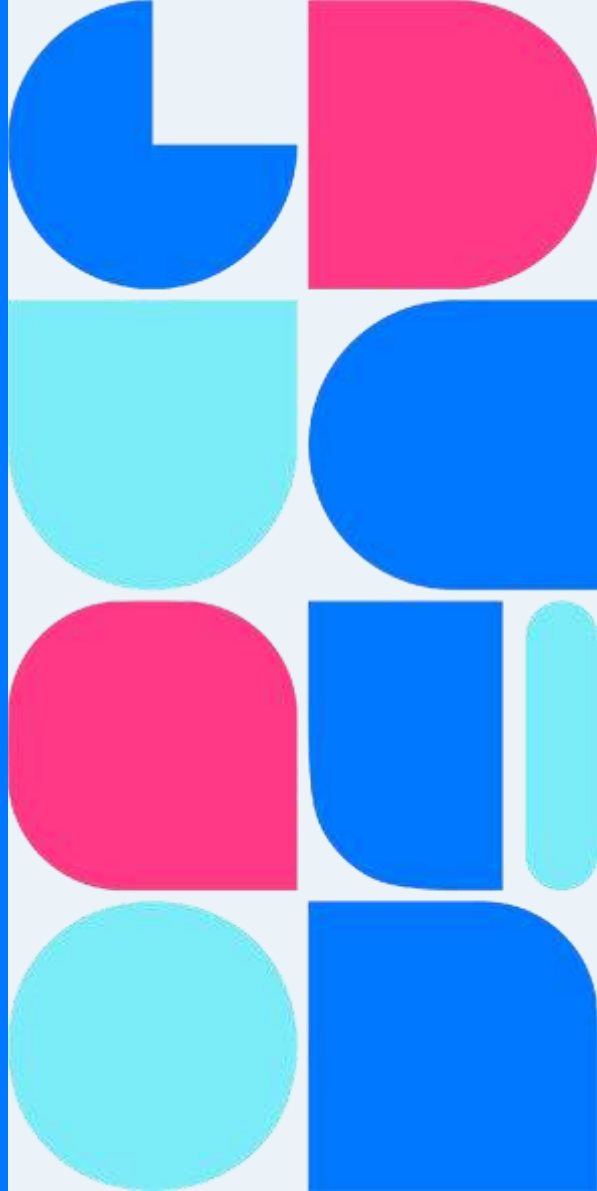
Документ	УНИВЕРСИТЕТ	ИТМО	Tfidf
Московский физико-технический институт	—	—	0
Московский государственный университет	0.41	—	0.41
Университет ИТМО	0.41	1.10	1.50

Вопрос

Документ «Университет университет университет» получит 1.23. Выше МГУ, но ниже ИТМО — правильно ли это?

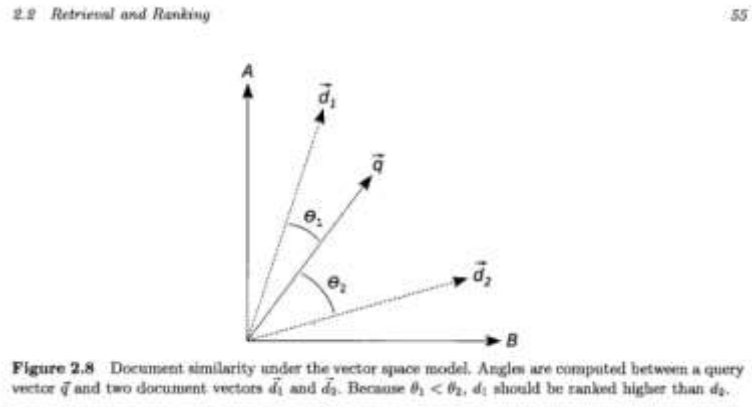
Модель векторного пространства

Обобщить TF-IDF: запросы и документы как векторы, релевантность как угол между ними



Модель векторного пространства (VSM)

- Каждому термину словаря — своя ось, |V| ~ 10^5–10^6 осей
- Документ и запрос — векторы весов терминов (например, TF-IDF)
- Скор — скалярное произведение или косинус угла
- Salton, Wong, Yang, 1975; система SMART (через 2 слайда)
- TfIdfScore — частный случай: вектор запроса из нулей и единиц



Косинус угла между запросом q и документами d_1, d_2 (Büttcher et al.)

$$\cos(q, d) = (q \cdot d) / (\|q\| \cdot \|d\|)$$

Векторы разреженные

	Terms		*Documents*
T1:	Bab(y,ies,y's)	D1:	Infant & Toddler First Aid
T2:	Child(ren's)	D2:	Babies & Children's Room (For Your Home)
T3:	Guide	D3:	Child Safety at Home
T4:	Health	D4:	Your Baby's Health and Safety: From Infant to Toddler
T5:	Home		
T6:	Infant	D5:	Baby Proofing Basics
T7:	Proofing	D6:	Your Guide to Easy Rust Proofing
T8:	Safety	D7:	Beanie Babies Collector's Guide
T9:	Toddler		

The 9×7 term-by-document matrix before normalization, where the element $\hat{a}_{ij}$ is the number of times term i appears in document title j:

$$\hat{A} = \begin{pmatrix} 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$

The 9×7 term-by-document matrix with unit columns:

$$A = \begin{pmatrix} 0 & 0.5774 & 0 & 0.4472 & 0.7071 & 0 & 0.7071 \\ 0 & 0.5774 & 0.5774 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.7071 & 0.7071 \\ 0 & 0 & 0 & 0.4472 & 0 & 0 & 0 \\ 0 & 0.5774 & 0.5774 & 0 & 0 & 0 & 0 \\ 0.7071 & 0 & 0 & 0.4472 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.7071 & 0.7071 & 0 \\ 0 & 0 & 0.5774 & 0.4472 & 0 & 0 & 0 \\ 0.7071 & 0 & 0 & 0.4472 & 0 & 0 & 0 \end{pmatrix}$$

- В документе ~10–10^3 разных слов из словаря в 10^5–10^6
- Храним только ненулевые компоненты — это и есть обратный индекс, развёрнутый по-другому
- Косинус считается только по общим терминам: остальные слагаемые нулевые
- В Python: scipy.sparse, TfidfVectorizer отдаёт CSR-матрицу

Матрица термин-документ из 7 документов и 9 терминов уже почти вся из нулей (Berry & Browne)

Варианты TF-IDF: нотация SMART

- SMART — экспериментальная поисковая система Джерарда Солтона (Корнелл, с 1960-х)
- На ней появились VSM, TF-IDF-взвешивание и Rocchio
- Солтон и Бакли (1988) сравнили десятки схем взвешивания и ввели короткую запись
- Вес термина = (вариант TF) × (вариант DF) × (нормировка) — по букве на каждый множитель

Как читать код: три буквы для документа, точка, три буквы для запроса

Документ: lnc

l	n	c
log TF	без IDF	косинус

Запрос: ltc

l	t	c
log TF	IDF	косинус

Таблица SMART

Выбираем по одной букве из каждой колонки — получаем схему взвешивания

Частота термина (TF)		Документная частота (DF)		Нормировка	
n	*natural* TF	n	*no* 1	n	*none* 1
l	*logarithm* $1 + \log TF$	t	*idf* $\log N / DF$	c	*cosine* $1 / \|w\|$ (длина вектора)
a	*augmented* $0.5 + 0.5 \cdot TF / \max TF$	p	*prob idf* $\max(0, \log(N - DF) / DF)$	u	*pivoted unique* $1 / u$ (pivoted, дальше в лекции)
b	*boolean* 1, если $TF > 0$, иначе 0			b	*byte size* $1 / CharLength^{\alpha}$, $\alpha < 1$
L	*log average* $(1 + \log TF) / (1 + \log avg\ TF)$				

В sklearn: sublinear_tf=True — буква l (log TF), use_idf=True/False — «t» / «n», norm='l2' — «c» (косинус)

Нормализация по длине

Длинный документ содержит больше слов, в том числе слов запроса. Без нормировки он всегда выигрывает

Без нормировки

1. Длинные документы выигрывают просто за счёт объёма
2. Двойная копия документа получает двойной скор

Косинус (L2-нормировка)

1. Скор не зависит от «удвоения» текста
2. Но короткие документы теперь выигрывают слишком часто

Singhal, Buckley, Mitra (SIGIR 1996)

Сравнили вероятность документа попасть в выдачу с вероятностью быть релевантным: при косинусе длинные документы релевантны чаще, чем попадают в топ

Косинус и короткие документы

Вклад слова запроса — w(t, d) / ‖d‖. А норма ‖d‖ растёт от ВСЕХ слов документа, не только от слов запроса.

Запрос «цунами». Вес слова = его tf, IDF для простоты не берём

Короткая справка: 4 разных слова	Статья про цунами: 100 разных слов, «цунами» 5 раз
$\|\|d\|\| = \sqrt{4} = 2$	$\|\|d\|\| = \sqrt{(5^2 + 99)} \approx 11.1$
скор = 1 / 2 = 0.50	**скор = 5 / 11.1 ≈ 0.45**

Справка, где слово просто упомянуто, обходит статью, которая целиком про цунами

Многословность

То же содержание, просто больше слов. Такую длину надо нормировать — это и делает косинус.

Охват

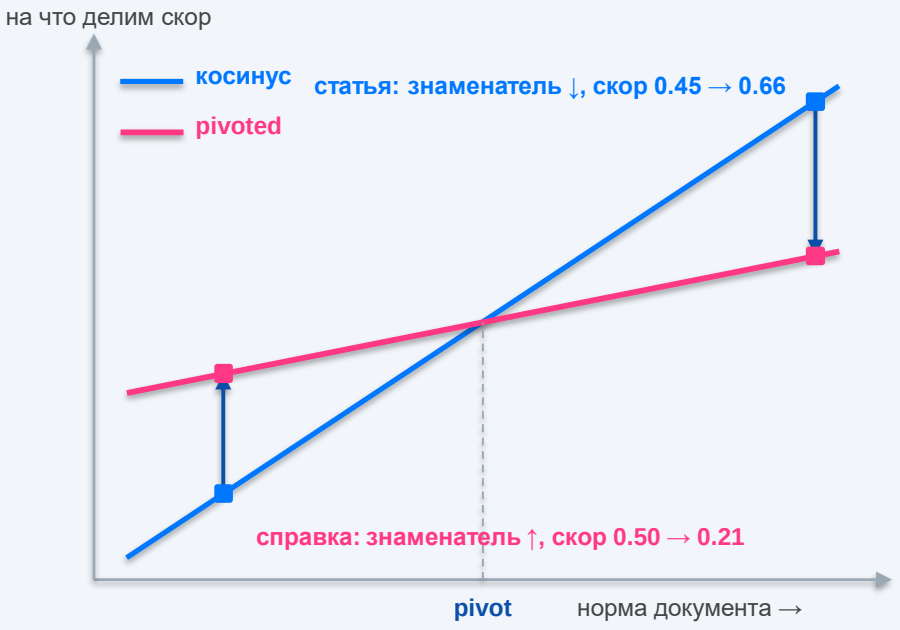
Больше содержания и тем. Такой документ чаще релевантен, штрафовать его не за что.

Косинус считает любую длину многословностью и перекорректирует. Лечение — pivoted normalization, дальше

Pivoted normalization

$$\text{норма}'(d) = (1 - s) \cdot \text{pivot} + s \cdot \text{норма}(d)$$

Делим скор не на норму документа, а на норма′. Это прямая, повёрнутая вокруг точки pivot



Пример с прошлого слайда

средняя норма в коллекции pivot = 6, наклон s = 0.3

	косинус	pivoted
справка	0.50	0.21
статья	0.45	**0.66**

Статья про цунами снова выше справки

Как выбирать s

- s = 1 — обычный косинус
- s = 0 — делим всех на pivot, нормировки нет
- подбирают на dev: у Singhal 0.2–0.3, на нашем семинаре лучшее s = 0.4

Аксиоматический подход

Fang, Tao, Zhai (SIGIR 2004): какие свойства должны быть у любой разумной формулы ранжирования

Аксиома	Смысл
TFC1	больше вхождений слова запроса → скор не ниже
TFC2	каждое следующее вхождение даёт меньший прирост
TDC	совпадение по редкому термину ценнее, чем по частому
LNC1	дописали в документ посторонние слова → скор не выше
LNC2	склеили документ сам с собой k раз → скор не ниже

Проверьте сами: какую аксиому нарушает TfIdfScore с «сырым» TF? А какую — TF-IDF без нормировки длины?

TF-IDF как векторизация текста

```
from sklearn.feature_extraction.text import (
    TfidfVectorizer)

vec = TfidfVectorizer(sublinear_tf=True)
X = vec.fit_transform(docs)      # CSR
q = vec.transform([query])
scores = (X @ q.T).toarray().ravel()
```

Косинус = скалярное произведение, если векторы уже L2-нормированы. Поэтому X @ q.T — это и есть ранжирование по косинусу

- По умолчанию: сырой TF, сглаженный IDF, L2-нормировка
- То же представление — стандартный первый бейзлайн для классификации и кластеризации текстов
- Для русского свой анализатор с лемматизацией: на семинаре

Ограничение: мешок слов

TF-IDF и вся VSM не видят порядок и близость слов

«МФТИ лучше, чем МГУ»

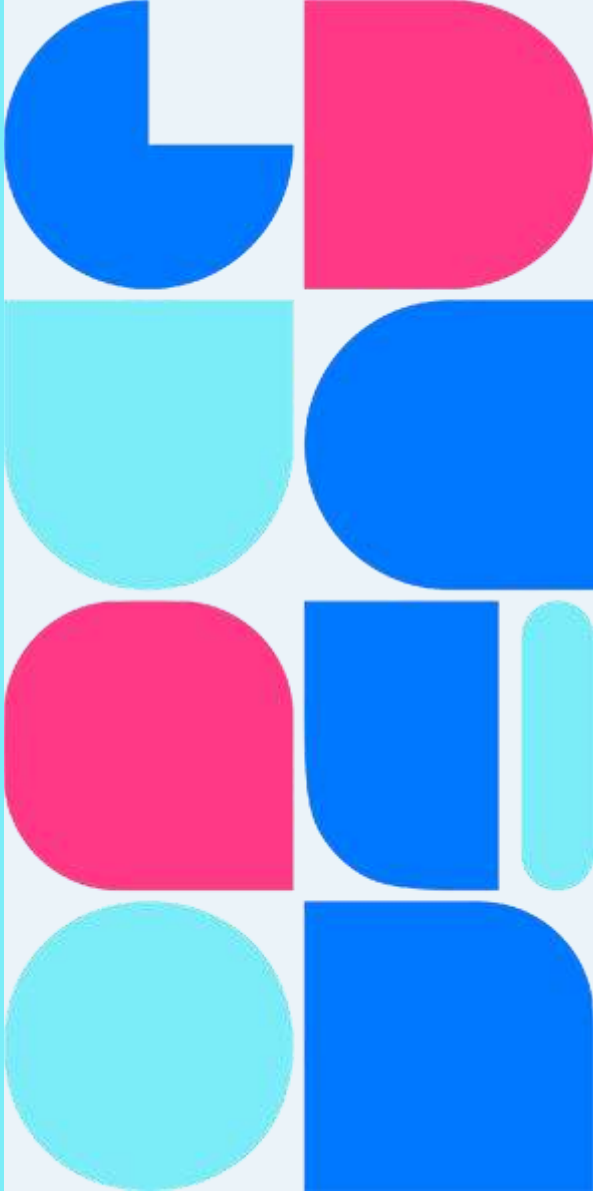
«МГУ лучше, чем МФТИ»

одинаковые векторы → одинаковый скор по любому запросу

- Запрос «нью йорк»: документ, где «нью» в начале, а «йорк» в конце, получит полный скор
- Лечение — координатный индекс из лекции 2: учитываем близость, фразы, пассажи
- Отдельно учитываем зоны: слово в заголовке важнее, чем в тексте (BM25F)
- Всё это — часть II

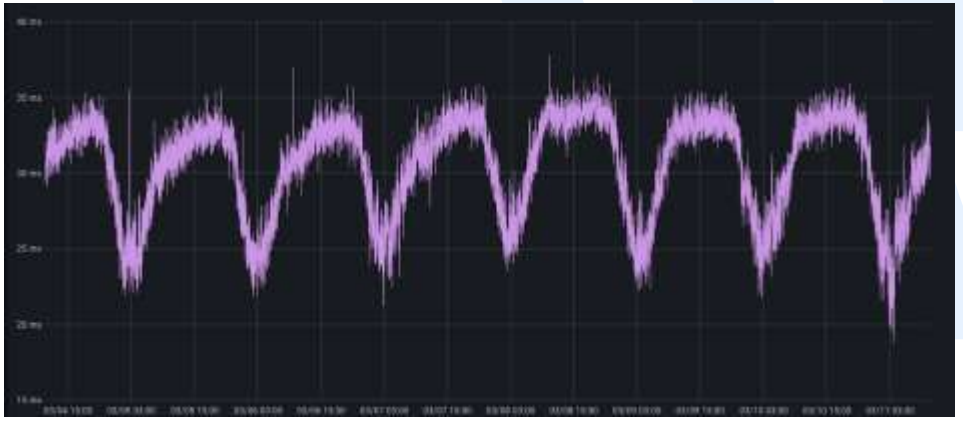
Ранжирование на обратном индексе

Посчитать топ-10 по миллиарду документов за десятки миллисекунд



TF-IDF на миллиарде документов

- Корпус ~10^9 документов — обычный масштаб для VK
- Посчитать скор для всех пар (q, d) — секунды на один запрос
- За 35 мс нужно найти кандидатов, посчитать ~1000 признаков и применить модель
- Нужно не считать то, что заведомо не попадёт в топ



Время ответа бэкенда поиска VK Видео: даже в час пик около 35 мс

Что храним в индексе

TfIdf(q, d) = 0, если в d нет слов запроса. Значит, скор нужен только документам из постингов запроса.

Постинг теперь — пара (docID, tf)

московский	(1, 1) (2, 1)
университет	(2, 1) (3, 1)
итмо	(3, 1)
физтех	(17, 107) (42, 3) …

В игрушечном корпусе tf = 1. В статье про МФТИ «физтех» — 107 раз

Офлайн, при индексации

- tf — в постинге
- DF — длина постинг-листа, IDF — из неё
- нормы документов — для косинуса

Онлайн, на запрос

только умножить и сложить по постингам слов запроса

Зачем tf в постинге

Он входит в формулу: tf · IDF. Без него пришлось бы в момент запроса открывать сам документ и считать вхождения — а это ровно то, от чего уходим. Для булева поиска хватало docID, для ранжирования нужно «сколько раз».

Term-at-a-time: пошагово

Запрос «университет итмо». Берём один термин, проходим его постинг целиком, потом следующий.

Шаг 1: «университет»

IDF = 0.41 · постинг:

`(2, 1)` `(3, 1)`

аккумулятор

1 МФТИ	—
2 МГУ	**0.41**
3 ИТМО	**0.41**

Шаг 2: «итмо»

IDF = 1.10 · постинг:

`(3, 1)`

аккумулятор

1 МФТИ	—
2 МГУ	0.41
3 ИТМО	**1.51**

Итог: сортируем

1.	**ИТМО**	1.51
2.	**МГУ**	0.41

МФТИ не тронули ни разу: его нет ни в одном постинге запроса.

Это и есть «считаем только кандидатов».

Слабое место: аккумуляторы

Скор окончательный только после последнего термина. До этого храним частичный скор каждого встреченного документа, а частое слово вроде «год» тянет за собой миллионы.

Кворум поверх TAAT

В том же проходе TAAT рядом со скором копим сумму IDF совпавших слов. Не набрал порог — не кандидат.

Запрос «что есть вечером», порог 9

IDF(ЧТО) = 2	IDF(ЕСТЬ) = 5	IDF(ВЕЧЕРОМ) = 10

одна запись аккумулятора — два числа:

```
для (doc, tf) в постинге t:
    acc[doc].score   += tf · IDF(t)
    acc[doc].idf_sum += IDF(t)
```

Аккумулятор после всех терминов

документ	score	Σ IDF	
d_1 «что» ×5, «есть» ×2	**20**	**7**	**отсеян**
d_2 «есть», «вечером»	15	**15**	**кандидат**

d_1 выигрывает по скору, но отсеян

Скор 20 он набрал повторами служебного «что». Самого важного слова «вечером» в нём нет: 7 < 9.

Что это даёт, а что нет

Проход по постингам не экономит. Экономит следующие этажи — дорогую формулу и LTR.

На семинаре, порог 0.3

кандидатов 3628 → 93, а MRR@10 вырос: 0.468 → 0.489

Топ-K: куча и порог θ

Нужны только K лучших. Держим min-кучу размера K: в корне — худший из текущих лучших.

K = 3, приходят новые документы



Пришёл 0.80 > θ — заменяет корень, θ растёт.
Пришёл 0.50 < θ — отбрасываем, кучу не трогаем.

Как это стыкуется с аккумулятором в TAAT:

словарь N записей → кворум → куча K — в самом конце

Зачем она

- O(N · log K) вместо полной сортировки, память O(K). В Python — heapq.nlargest
- В TAAT куча — просто сортировка в самом конце, работы почти не экономит
- Главное — побочный продукт: корень кучи θ. Документ со скором ниже θ в топ уже не попадёт
- По ходу TAAT кучу держать нельзя: скор ещё частичный, выкинутый документ мог бы потом вырасти

В TAAT порогом θ воспользоваться нельзя: скор готов только в конце. Чтобы θ отсекал, нужен другой порядок обхода

Document-at-a-time: пошагово

Курсор на каждый постинг, списки отсортированы по docID. Берём наименьший docID под курсорами.

Шаг 1: курсоры на 2 и 3 → берём 2

университет	2	3
итмо	3	

2 есть только в «университет»
скор = 0.41, готов → в кучу

Шаг 2: курсоры на 3 и 3 → берём 3

университет	2	3
итмо	3	

3 в обоих списках
скор = 1.51, готов → в кучу

Скор окончательный сразу: 2 уже не встретится ни в одном списке

TAAT: две проблемы

- ! память: запись на каждый документ
- ! работа: θ не помогает, считаем всех

Что меняет DAAT

- \+ в памяти только куча K и курсоры
- – сам работу не экономит
- → но скор готов сразу — можно отсекать

DAAT — не цель, а условие: постингов он читает столько же, но без него отсечка по θ невозможна. Ради неё и переходим — следующий слайд

Верхние границы: MaxScore и WAND

UB(t) — максимальный вклад термина t в скор любого документа. Считается при индексации, как IDF.

Запрос «нобелевская премия год», θ = 2.5

нобелевская	UB = 2.0
премия	UB = 1.2
год	UB = 0.3

миллионы постингов

Под курсорами документ, ещё до подсчёта скора:

«премия» + «год»: 1.2 + 0.3 = 1.5 < θ → не считаем

«нобелевская» + «премия»: 3.2 ≥ θ → считаем честно

θ — корень кучи сейчас: появляется после K документов и растёт

MaxScore (Turtle & Flood, 1995)

«год» один до θ не дотянет — он несущественный. Кандидатов берём только из списков существенных терминов, а в «год» заглядываем точечно

WAND (Broder et al., 2003)

Курсоры сортируем по docID и ищем pivot — первый документ, где сумма UB ≥ θ. Всё до него перепрыгиваем

Отсечка точная, не приближённая: UB — честный максимум, поэтому топ-K ровно тот же, что при полном подсчёте

Прыгают по постингам skip-листами из лекции 2 — вот зачем они были нужны

Block-Max WAND в Lucene

- Ding & Suel (SIGIR 2011): UB храним не на весь список, а на каждый блок из ~128 постингов
- Если максимум блока не дотягивает до θ — пропускаем блок целиком, не распаковывая
- Lucene 8 / Elasticsearch 7 (2019): включён по умолчанию
- Ускорение дизъюнктивных запросов от десятков процентов до ~13 раз
- Цена: движок больше не знает точное число найденных документов

Что видит пользователь ES

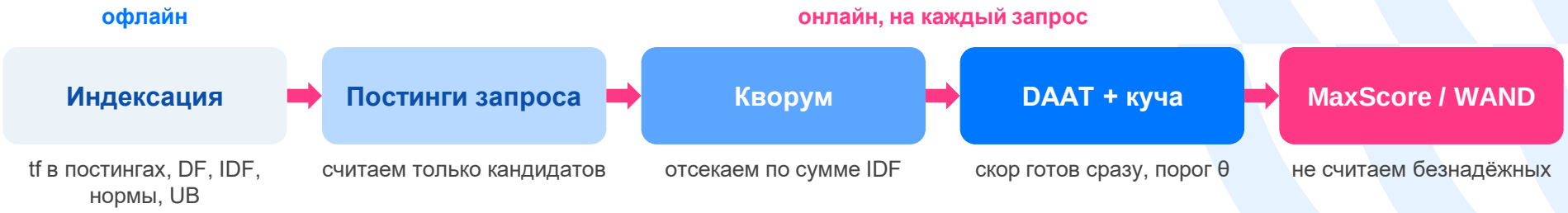
10 000+ hits

Точный счётчик отключили ради скорости: track_total_hits по умолчанию считает только до 10 000

С 2022 года Lucene для дизъюнкций перешёл на block-max MaxScore — идеи те же

Собираем вместе

Каждый шаг экономит на предыдущем. Так устроены Lucene, Elasticsearch и большие поисковики.



Отвечают на разные вопросы

- Кворум — кого считать кандидатом
- Куча, DAAT, WAND — как быстро найти среди них топ-K

На выходе

Топ-K черновым скором — и дальше на следующий этаж пирамиды ранжирования: к признакам и Learning to Rank

Блок-версия WAND — предыдущий слайд — это то же самое, только UB хранится на каждый блок постингов

Предобработка решает: русский язык

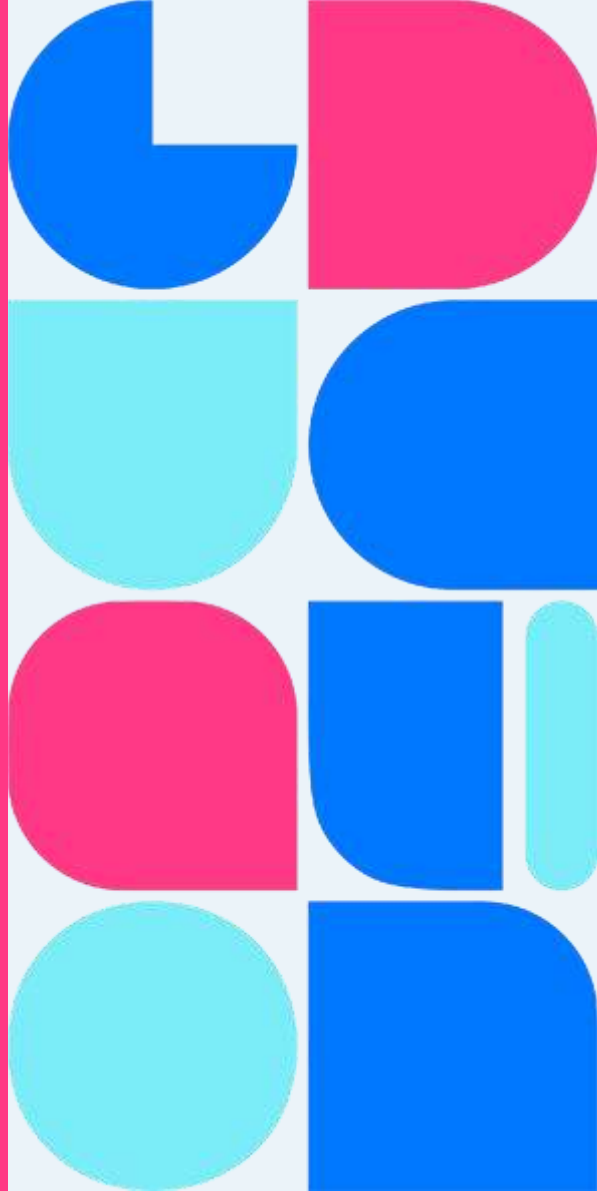
Dolamic & Savoy (JASIST 2009): коллекция CLEF, 94 запроса, шесть моделей ранжирования

Что сравнивали	Результат для русского
Стемминг vs без нормализации	MAP почти вдвое выше (для английского ~+4%)
Лёгкий vs агрессивный стеммер	статистически не различаются
Символьные 4-граммы vs стеммер	отстают всего на ~4%, без словарей
Удаление стоп-слов	около 2% разницы

Для морфологически богатых языков нормализация важнее выбора формулы. Сегодня стандарт — лемматизация (pymorphy3)

Лексический разрыв

Что делать, если в релевантном документе нет слов запроса



Lexical mismatch

Скор ненулевой только для общих терминов. Но смысл можно выразить разными словами

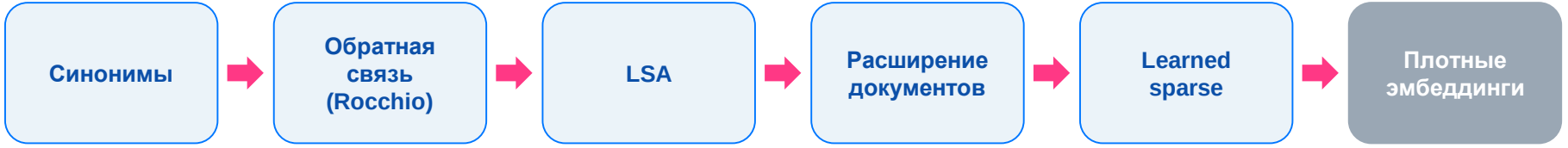
Синонимия

«дед мороз» и «санта клаус»
«подъезд» и «парадная»
→ теряем полноту

Полисемия

«ягуар»: животное, машина, напиток
«ключ»: от двери, родник, в музыке
→ теряем точность

Путь решений, от старых к новым



Расширение запроса синонимами

«московский международный конкурс юных музыкантов» превращается в

(МОСКОВСКИЙ | МОСКВА) (МЕЖДУНАРОДНЫЙ | ВСЕМИРНЫЙ | МИРОВОЙ) (КОНКУРС | ОЛИМПИАДА | СОСТЯЗАНИЕ) (ЮНЫЙ | МОЛОДОЙ | ЮВЕНИЛЬНЫЙ) (МУЗЫКАНТ | ПИАНИСТ | САКСОФОНИСТ)

Работает

1. Простые случаи: ПОДЪЕЗД = ПАРАДНАЯ
2. Ложится на булев поиск через ИЛИ
3. Растёт полнота

Проблемы

1. Где остановиться: пианист — синоним музыканта?
2. Какие веса дать синонимам
3. Словарь не масштабируется

Повторение из лекции 1

Обратная связь: алгоритм Rocchio

Идея VSM: сдвинуть вектор запроса к релевантным документам и от нерелевантных

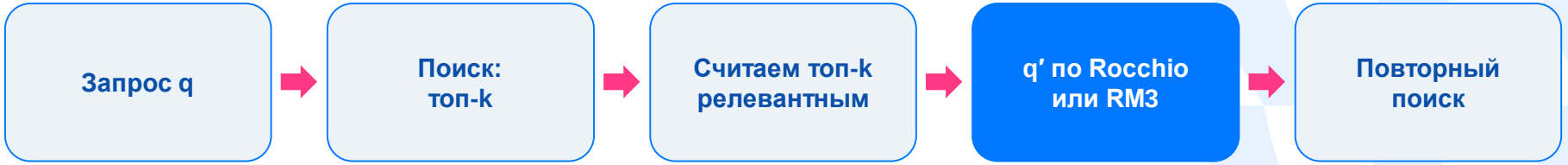
$$q' = \alpha \cdot q + \beta \cdot \text{mean}(d \in D^{+}) - \gamma \cdot \text{mean}(d \in D^{-})$$

- Rocchio, 1971, система SMART
- В q′ появляются новые слова — из релевантных документов
- Обычно γ маленькое или 0: отрицательные примеры помогают слабо

Откуда взять D^{+}?

1. Отметил пользователь (редко)
2. Клики (лекция про поведение)
3. Взять топ-k выдачи «вслепую» →

Псевдорелевантная обратная связь (PRF)



Плюсы

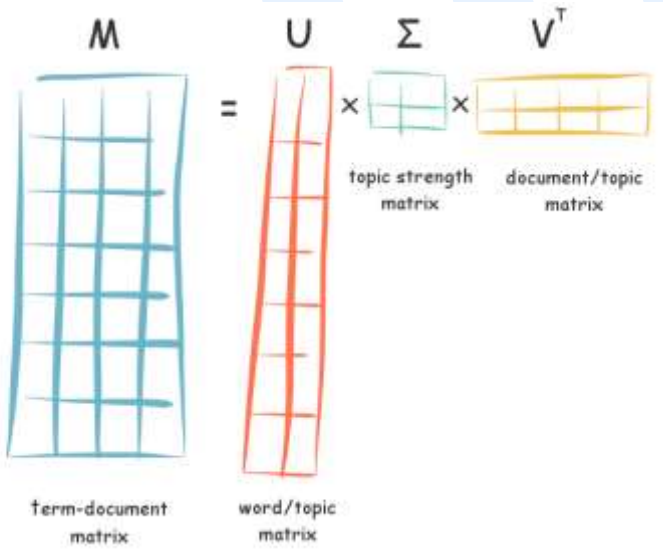
1. Никакой разметки не нужно
2. Лечит синонимию «данными»
3. RM3 — стандартный бейзлайн в Anserini/Pyserini

Минусы

1. Два прохода поиска: дороже
2. Query drift: плохой топ уводит запрос в сторону
3. Чаще помогает полноте, чем точности топа

LSA: семантика через разложение матрицы

- Дистрибутивная гипотеза: слова из похожих контекстов близки по смыслу
- Раскладываем матрицу термин-документ: $X \approx U_k \Sigma_k V_k^T$ (усечённое SVD)
- Оставляем $k \approx 100–300$ компонент — «скрытых тем»
- Документ и запрос становятся плотными векторами длины k
- Синонимы попадают в одни и те же темы: «дед мороз» ≈ «санта клаус»



Матрица термин-документ = термины×темы · сила тем · темы×документы

Deerwester, Dumais et al., 1990. Детали SVD — в книге Маннинга, гл. 18

LSI в поиске: почему не прижилось

Рецепт LSI

1. Индексация: SVD матрицы X, храним U_k, Σ_k
2. Запрос: $\hat{q} = \Sigma_k^{-1} U_k^T q$
3. Скор: косинус $\hat{q}$ и векторов документов

Почему не взлетело

1. SVD на 10^9 документов не посчитать
2. Плотные векторы не лягут в обратный индекс
3. Точный лексический матч становится хуже

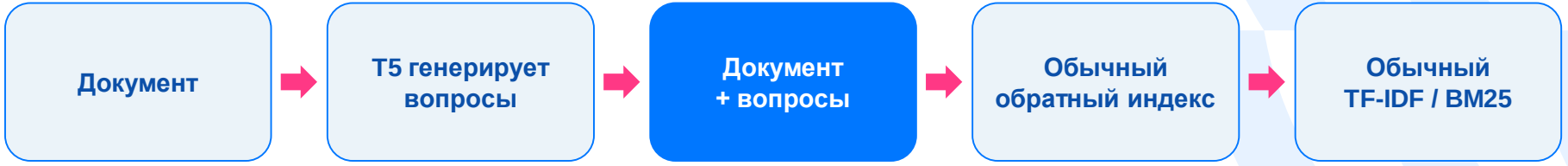
Что осталось от LSA

Лучше всего работает гибрид: $\alpha \cdot \text{TfIdf} + (1 - \alpha) \cdot \text{LSI}$

Сама идея «плотных векторов смысла» → word2vec (кстати, это тоже неявное разложение матрицы), DSSM, BERT — лекции 9–11

Расширяем документ, а не запрос

doc2query (Nogueira et al., 2019): модель генерирует вопросы, на которые отвечает документ



Почему это удобно

1. Вся нейросеть работает офлайн, при индексации
2. Поиск не меняется и остаётся быстрым
3. Повторы важных слов поднимают их TF

MS MARCO, MRR@10

BM25	**0.187**
docT5query	0.277

Learned sparse: TF-IDF с выученными весами

- Вес термина в документе предсказывает нейросеть, а не формула TF · IDF
- Модель может добавить в документ слова, которых там нет (расширение)
- Результат — всё ещё разреженный вектор → всё ещё обратный индекс
- Эволюция: DeepCT → DeepImpact → uniCOIL → SPLADE (2019–2024)
- В проде: Elastic ELSER, OpenSearch neural sparse, sparse-векторы в Qdrant

Подвох для WAND

У выученных весов другое распределение, чем у TF-IDF: верхние границы получаются «рыхлыми», и WAND пропускает меньше документов. Отсюда отдельная линия статей про быстрый поиск по learned sparse

Тизер: обучение таких моделей разберём в нейросетевом блоке курса

Расширение запроса с помощью LLM

query2doc (EMNLP 2023)

1. LLM пишет псевдо-документ-ответ
2. Приклеиваем его к запросу
3. BM25 +3…15% на MS MARCO и TREC DL

HyDE (ACL 2023)

1. LLM пишет гипотетический документ
2. Ищем по нему, а не по запросу
3. Похоже на Rocchio, но без первого поиска

Оговорка

Weller et al. (Findings EACL 2024): расширения запроса помогают слабым ретриверам и обычно вредят сильным. Плюс LLM на каждый запрос — это секунды и деньги

Что работает сегодня

Бенчмарк	Лексический поиск	Нейросети / гибрид
BEIR, 18 датасетов, zero-shot	BM25 — «надёжный бейзлайн»	плотные модели 2021 года чаще проигрывали
MIRACL ru, nDCG@10	BM25: 0.334	mDPR: 0.407, гибрид: 0.532
Длинные документы (RusBEIR)	BM25 выигрывает	лимит в 512 токенов мешает

Гибрид через Reciprocal Rank Fusion (Cormack et al., SIGIR 2009)

$$\mathrm{RRF}(d) = \sum 1 / (k + \mathrm{rank}_i(d)), \quad k = 60$$

Нужны только ранги, нормировать скоры не надо. В Elasticsearch — rrf retriever с версии 8.16

Итоги

- ✓ Качество ранжирования меряют на тестовой коллекции: P@K, Recall@K, MRR
- ✓ TF-IDF = локальная важность слова (TF) × глобальная информативность (IDF)
- ✓ VSM: запрос и документ — векторы, скор — косинус. Нормировка длины и сублинейный TF обязательны
- ✓ Аксиомы TFC / TDC / LNC помогают рассуждать о формулах без экспериментов
- ✓ Быстрый топ-K: только кандидаты из индекса, куча, MaxScore / WAND / Block-Max WAND
- ✓ Лексический разрыв: PRF, LSA, расширение документов, learned sparse, гибрид

Дальше: часть II

Вероятностные модели

1. Принцип вероятностного ранжирования
2. BIM и откуда в IDF логарифм
3. BM25 и BM25F: насыщение, длина, зоны
4. Языковые модели для поиска и сглаживание (привет, лекция 3)

Метрики и близость слов

1. Градуированная релевантность, DCG и nDCG, MAP
2. Асессоры и LLM-оценка
3. Координатный индекс в деле: близость, пассажи
4. ДЗ 2: контест на Kaggle

Материалы

Почитать

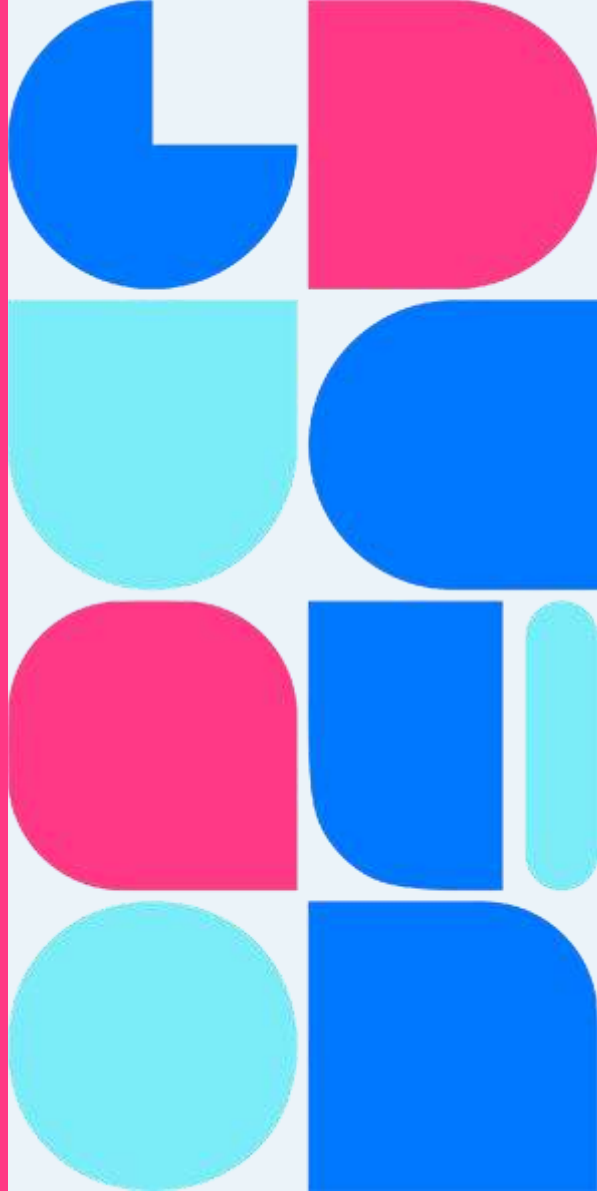
1. Маннинг и др. «Введение в информационный поиск»: гл. 6 (VSM), 7 (быстрый топ-K), 9 (Rocchio), 18 (LSA)
2. Singhal, Buckley, Mitra. Pivoted Document Length Normalization, SIGIR 1996
3. Fang, Tao, Zhai. A Formal Study of IR Heuristics, SIGIR 2004
4. Lin, Nogueira, Yates. Pretrained Transformers for Text Ranking, гл. 1–2

Потрогать руками

1. scikit-learn: TfidfVectorizer, TruncatedSVD
2. Pyserini / Anserini: воспроизводимые BM25, RM3, SPLADE
3. PISA: исследовательский движок, все DAAT-алгоритмы
4. Блог Elastic: Faster retrieval with Block-Max WAND
5. RusBEIR и ruMTEB: русские бенчмарки поиска

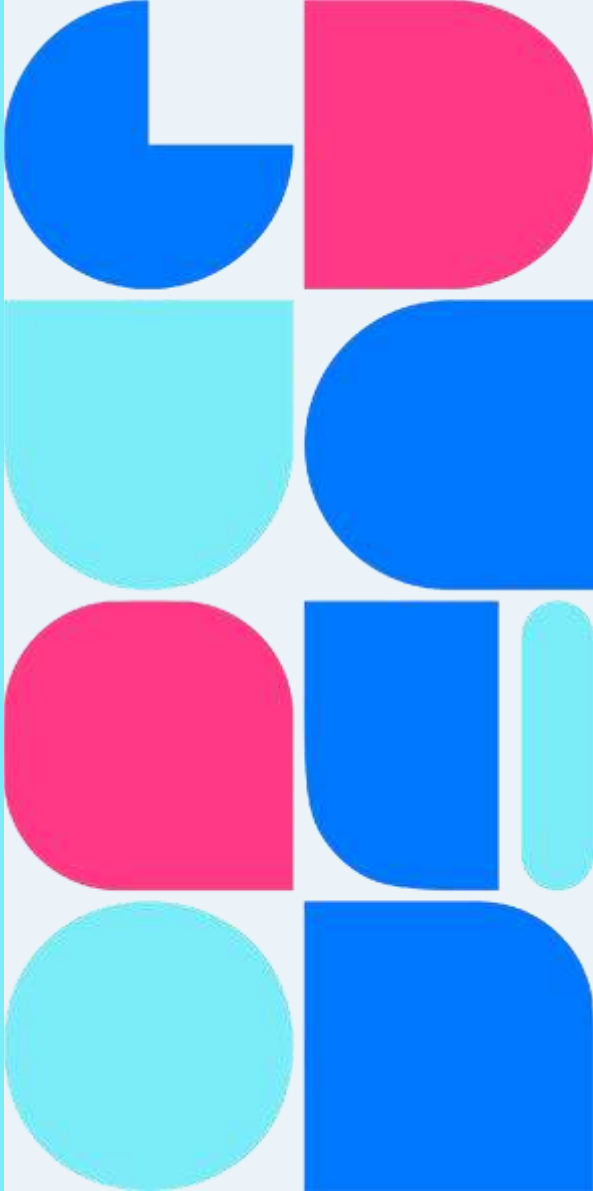
Перерыв

10 минут, потом семинар



Семинар

TF-IDF руками, векторная модель в sklearn, лемматизация, кворум, whoosh, PRF и LSA на русских вопросах к Википедии (RuBQ)



Спасибо
за внимание!

Руслан Аллаяров

education